Projeto Final da Disciplina IN1102 Aprendizagem de Máquina

Pós-graduação, Centro de Informática (CIn/UFPE)

Prof. Leandro Maciel Almeida

2026.2 (release 2026.2.1)

1 Objetivo Geral

Este projeto visa proporcionar aos alunos uma experiência prática aprofundada em modelos de classificação supervisionada recentes e avançados, incluindo a análise detalhada do funcionamento, a comparação sistemática e a avaliação estatística rigorosa dos modelos selecionados sobre dados tabulares do benchmark *TabArena-v0.1* (NeurIPS 2025).

2 Descrição do Trabalho

Os grupos têm o mesmo tamanho definido na primeira parte da disciplina. Cada grupo terá atribuído um modelo recente de classificação supervisionada (modelo tema) para estudo e implementação. Após a análise do modelo sugerido, o grupo pode apresentar outras opções de modelos recentes com a devida justificativa. Uma única solicitação de mudança por grupo será aceita para deferimento ou não. O projeto final da disciplina compreende duas etapas: o entendimento teórico do modelo atribuído e a realização de experimentos comparativos. Para ambas as etapas devem ser apresentadas as seguintes informações:

Tempo de apresentação: cada grupo terá um tempo total de **20 minutos** para apresentar o trabalho, organizado em dois blocos contíguos de 10 minutos: o primeiro dedicado à **Etapa 1** (apresentação teórica do modelo atribuído) e o segundo dedicado à **Etapa 2** (resultados experimentais, comparações, testes estatísticos e conclusões). O tempo será controlado pelo professor. A pontuação do tempo segue a fórmula contínua da rubrica (janela de 20 min no total, sem tolerância, e de 10 ± 1 min em cada bloco).

2.1 Etapa 1: Apresentação Teórica do Modelo

Nos primeiros 10 minutos do seminário, cada grupo apresentará o modelo atribuído. A apresentação deve contemplar:

- Motivação e contexto histórico do modelo;
- Funcionamento detalhado do modelo (arquitetura, hiperparâmetros principais, peculiaridades e funcionamento interno);
- Forma de aprendizado e representação dos padrões nos dados;
- Complexidade computacional e requisitos de hardware;

- Aplicações práticas e limitações.

2.2 Etapa 2: Experimentos Comparativos

Nesta etapa, cada grupo realizará experimentos comparando o seu modelo com três modelos de *gradient boosting* amplamente utilizados (*LightGBM*, *XGBoost* e *CatBoost*). O modelo do grupo é avaliado em duas variantes: com a configuração padrão recomendada pelos autores (`group_default`) e após busca de hiperparâmetros (`group_tuned`). Os três baselines usam a configuração padrão TD (*tuned defaults*) do pacote `pytabkit` (hiperparâmetros obtidos por meta-aprendizado; Holzmüller et al., NeurIPS 2024), sem busca.

Bases de Dados: Serão utilizados **30 datasets**, escolhidos **pelo grupo** entre os 38 datasets de classificação do benchmark **TabArena-v0.1** (suíte 457 do OpenML), curado manualmente para garantir relevância prática, estrutura tabular IID e licenciamento adequado. Sendo n o número total de instâncias do dataset, a seleção deve ser estratificada por regime de tamanho:

- **10 datasets de regime pequeno** ($n < 2.500$; o catálogo oferece 12);
- **10 datasets de regime médio** ($2.500 \leq n < 10.000$; o catálogo oferece exatamente 10, que são, portanto, comuns a todos os grupos);
- **10 datasets de regime grande** ($n \geq 10.000$; o catálogo oferece 16).

Recomenda-se que cada regime contenha pelo menos um problema de classificação binária e um de classificação multiclasse, sempre que o catálogo oferecer ambos. A seleção é registrada no arquivo `data/selecao_datasets.txt` do repositório e conferida pelo validador `python -m src.pipeline.check_selection`, que acusa erro para *task* fora do catálogo, repetida ou contagem por regime diferente de 10, e emite aviso quando falta problema binário ou multiclasse em um regime. A seleção é feita a partir das características do catálogo (tamanho, classes, atributos, ausentes), **antes de qualquer resultado experimental**, e fica congelada no `protocolo.lock`. A justificativa da seleção (critérios usados para escolher 10 entre 12 pequenos e 10 entre 16 grandes) é parte avaliada do relatório, assim como a tabela dos datasets escolhidos contendo: nome do dataset, identificador da *task* no OpenML, número de instâncias (n), número de atributos, número de classes, proporção de atributos categóricos, presença de valores ausentes e regime atribuído.

Protocolo de Avaliação: cada dataset é avaliado nas **partições oficiais** das *tasks* do OpenML usadas pelo TabArena (repetição 0, 3 *folds* estratificados), que formam a **validação cruzada externa**. Toda decisão (busca de hiperparâmetros e estudos) é tomada por **validação cruzada interna** dentro do treino de cada *fold* externo, e o teste de cada *fold* externo é usado **uma única vez**, na avaliação final, depois do congelamento do protocolo. O protocolo completo, o orçamento de busca e os comandos estão na Seção 3. Os baselines e os modelos do `pytabkit` recebem imputação de valores ausentes numéricos pela mediana do treino com indicador binário de ausência; essa decisão de protocolo deve ser mencionada no relatório.

Métricas: todas derivadas de `predict_proba`:

- AUC OvO (*One-vs-One*), métrica primária da análise estatística;
- Acurácia (ACC);
- G-Mean (média geométrica dos *recalls* por classe);

- Entropia cruzada (CE);
- Custo computacional em segundos, decomposto em tempo de busca, de treino e de predição.

Framework de Comparação Estatística: Deverá ser seguido o protocolo de comparação de classificadores em múltiplos datasets de **Demšar (2006)**, implementado pelo próprio grupo e aplicado ao arquivo `results/test.csv` da avaliação final:

1. **Uma medida por dataset**: a média da métrica nos testes dos 3 *folds* externos. Os *folds* de um mesmo dataset não são observações independentes e não entram como tal nos testes. Entram na análise apenas os datasets em que todos os sistemas comparados concluíram todos os *folds* (regra de completude da infraestrutura).
2. **Postos por dataset** (1 = melhor; empates recebem o posto médio) e postos médios R_j.
3. **Teste de Friedman** com a correção de **Iman-Davenport**; a decisão é tomada pela estatística F_F, com $k-1$ e $(k-1)(N-1)$ graus de liberdade. No teste principal, $k = 4$ (`group_tuned`, LightGBM, XGBoost e CatBoost) e $N \leq 30$.
4. **Modelo do grupo como controle** (`group_tuned`): comparação com cada baseline pelo procedimento de **Holm** aplicado às estatísticas z dos postos médios, e diagrama de **Bonferroni-Dunn**.
5. **Análise complementar todos contra todos**: teste de **Nemenyi** e diagrama de diferença crítica (CD). Para referência, com $k = 4$, $N = 30$ e $\alpha = 0{,}05$, a CD de Nemenyi é 0,856 e a de Bonferroni-Dunn é 0,798.
6. **Padrão vs. com busca**: teste de **Wilcoxon** (*signed-ranks*) entre `group_default` e `group_tuned` nos N datasets, relatando o número de diferenças efetivamente usado no teste (que pode ser menor que N quando há diferenças nulas, conforme a docstring de `wilcoxon_signed_ranks`). O N do Wilcoxon é o dos datasets em que `group_default` e `group_tuned` concluíram todos os *folds* e pode diferir do N do teste principal; relate os dois. Esse teste descreve o resultado final. Os cinco sistemas e o controle (`group_tuned`) são fixados a priori pelo enunciado; nenhum resultado, nem o da validação interna, é usado para escolhê-los.
7. **Robustez**: a função `wilcoxon_vs_control` (Wilcoxon do controle contra cada baseline, com ajuste de Holm) é parte obrigatória do módulo e é verificada pelos testes; é **opcional** apenas a discussão, no relatório, de eventuais divergências em relação ao item 4 (Benavoli, Corani e Mangili, 2016).

Os pós-testes baseados em postos médios (itens 4 e 5) só são interpretados como testes se o F_F de Iman-Davenport rejeitar a hipótese nula; caso contrário, as diferenças de postos, as estatísticas z e os diagramas são apresentados apenas como descrição, sem conclusão de diferença significativa. A Figura 1 resume a sequência de decisões da análise estatística.

Complementam a análise: (i) intervalos de confiança de 95% por *bootstrap* percentil sobre as predições de teste gravadas pela avaliação final, por dataset e *fold*, apresentados no apêndice do relatório; a média dos limites desses intervalos entre datasets **não** é um intervalo de confiança da média entre datasets, e o resumo entre datasets deve usar a mediana da largura dos intervalos; (ii) análise por regime, **apenas descritiva**, segmentando os resultados por tamanho (pequeno, médio, grande), número de classes (binário vs. multiclasse), proporção de atributos categóricos (baixa vs. alta, limiar de 25%) e presença de valores ausentes; (iii) análise

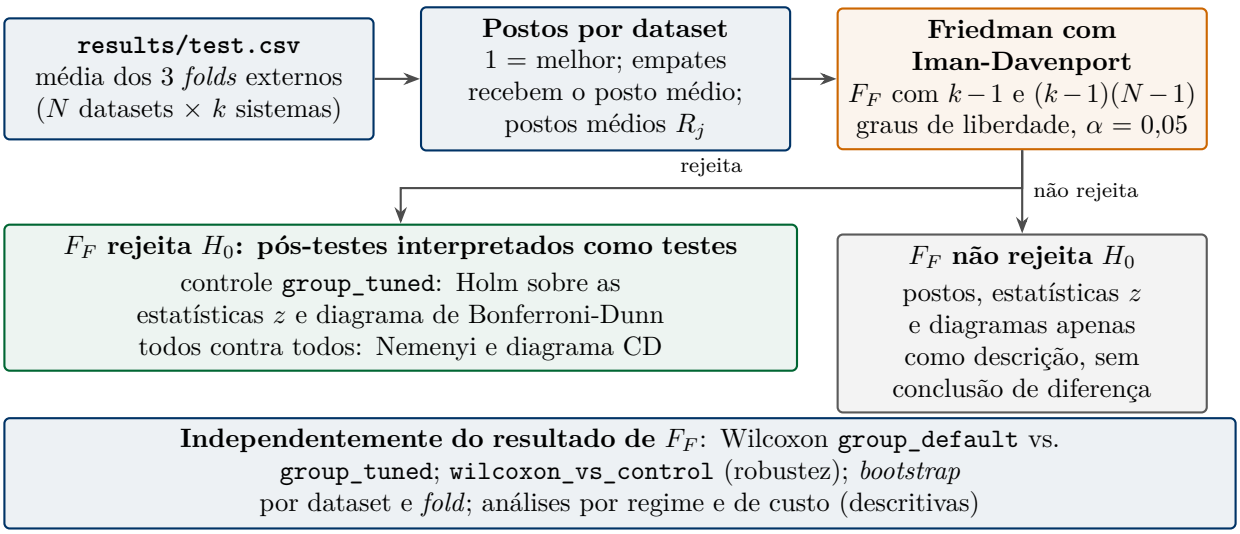


Figura 1: Sequência de decisões da análise estatística segundo Demšar (2006), aplicada à métrica primária (AUC OvO) e repetida para uma métrica secundária. Os pós-testes baseados em postos médios só são lidos como testes quando o F_F de Iman-Davenport rejeita a hipótese nula de equivalência.

de custo vs. desempenho; (iv) estudo de overfitting e viés-variância, obrigatório, feito na fase de desenvolvimento (Seção 4). Como **extensão opcional**, sem código fornecido, o grupo pode acrescentar a análise bayesiana de Benavoli, Corani, Demšar e Zaffalon (JMLR, 2017), com região de equivalência prática (ROPE) declarada antes da análise.

Procedimentos:

1. Escolher e justificar os 30 datasets a partir do catálogo, antes de qualquer experimento, e validá-los com `check_selection`;

2. Implementar o modelo tema como estimador compatível com *scikit-learn* e definir o espaço de busca, justificando *wrapper*, faixas e escalas **a priori**, com base no artigo original e na documentação do pacote. O teste de contrato verifica rótulos de texto, atributos categóricos (inclusive categoria não vista no treino), valores ausentes, multiclasse, probabilidades válidas, predição de uma única instância e reprodutibilidade;

3. Declarar, antes de olhar qualquer curva ou resultado, o subconjunto de datasets dos estudos e as grades de valores (Seção 3);

4. Executar a fase de desenvolvimento: `run_validation` com os cinco sistemas (`group_default`, `group_tuned`, LightGBM, XGBoost e CatBoost) nos 30 datasets, por validação cruzada interna no treino de cada *fold* externo;

5. Nos grupos cujo modelo tem API *scikit-learn* nativa, apresentar o estudo de sensibilidade e o estudo específico do modelo descritos no parágrafo *Equilíbrio de carga entre os modelos* (Seção 5), por validação cruzada interna;

6. Realizar o estudo de overfitting e viés-variância, incluindo os erros por classe nas predições de validação (Seção 4, *notebook* 05), e responder às suas perguntas no relatório;

7. Comitar todo o código e a seleção, congelar o protocolo (`freeze`), comitar o lock, executar a avaliação final **uma única vez** (`final_test`, *notebook* 06), comitar `results/` imediatamente e conferir o histórico com `check_protocol` (Seção 3.5);

8. Implementar o módulo de estatística segundo Demšar (2006), o *bootstrap*, a agregação por regime, as tabelas-resumo e os gráficos, validando cada componente pelos testes de aceitação (que usam dados sintéticos e podem ser executados a qualquer momento), e aplicá-los a `results/test.csv`;

9. Responder às perguntas do roteiro `notebooks/04_analise_estatistica.ipynb` no relatório, incluindo a repetição da análise para uma métrica secundária e a discussão da multiplicidade entre métricas;

10. Discutir as ameaças à validade do estudo (assimetria de busca entre o modelo do grupo e os baselines, imputação, subamostragem, falhas de execução, número de datasets por regime, otimismo da validação interna de `group_tuned`, estudos feitos em um único *fold* externo, origem das faixas do espaço de busca);

11. Documentar todo o *pipeline* em ambiente reproduzível (`pyproject.toml` e `uv.lock` com versões fixadas, sementes fixas, comandos de execução, `protocolo.lock`, `test_log.csv`, `test_manifest.json`, histórico git aprovado por `check_protocol` e, opcionalmente, `Dockerfile`);

12. Produzir um *model card* detalhado do modelo tema, seguindo o template em `model_cards/TEMPLATE.md` do repositório (seções inspiradas em Mitchell et al. (2019), com extensões da disciplina: detalhes do modelo, uso pretendido, fatores observados nos regimes, métricas com intervalo de confiança de 95%, dados de avaliação e protocolo, dados de treino e pré-treino, análise quantitativa segundo Demšar (2006), overfitting e viés-variância, considerações éticas, avisos e recomendações, reprodutibilidade e referências). Embora use os resultados da Etapa 2, o *model card* é avaliado no bloco da Etapa 1 (clareza e profundidade na explicação do modelo), conforme a rubrica.

Nos 10 minutos finais do seminário, o grupo apresentará todo o fluxo de trabalho da Etapa 2, incluindo o protocolo, o estudo de overfitting, comparações entre modelos, testes estatísticos e conclusões acerca do desempenho.

3 Protocolo experimental: treino, validação e teste

3.1 Os três papéis dos dados

Em aprendizagem de máquina, os dados disponíveis cumprem três papéis distintos (Hastie, Tibshirani e Friedman, 2009, Seção 7.2):

- o **treino** ajusta os parâmetros do modelo (pesos, árvores, funções de forma; nos modelos de contexto, o conjunto de contexto);
- a **validação** escolhe: hiperparâmetros, variantes do modelo, regras de pré-processamento e qualquer outra decisão de projeto;
- o **teste** estima o desempenho de generalização do sistema já escolhido e é usado **uma única vez**, ao final.

A estimativa no teste só é honesta se o teste não participou de nenhuma escolha. Se o teste for consultado para decidir algo, ele passa a fazer parte da validação e a estimativa final fica otimista, sem que haja outro conjunto independente para medir o erro. Quando os dados não permitem separar uma validação fixa grande, a validação é feita por validação cruzada dentro do treino; a regra de ouro é que **todas** as etapas que envolvem escolhas, inclusive a seleção de variáveis e o pré-processamento supervisionado, sejam refeitas dentro de cada partição de treino (Hastie, Tibshirani e Friedman, 2009, Seção 7.10.2).

3.2 Vazamento de dados

Há **vazamento de dados** (*data leakage*) sempre que informação do conjunto de teste influencia, direta ou indiretamente, o sistema avaliado. A seleção de modelo também pode sobreajustar a validação (Cawley e Talbot, 2010): quanto mais escolhas são feitas olhando um mesmo conjunto, mais otimista fica a estimativa nesse conjunto. Exemplos concretos deste projeto, todos proibidos:

- escolher hiperparâmetros, a configuração padrão ou a variante do *wrapper* olhando `results/test.csv` ou as predições de teste;
- definir ou rever a faixa de um hiperparâmetro do espaço de busca, ou a grade de um estudo de sensibilidade, depois de ver o desempenho no teste;
- ajustar a regra de subamostragem do contexto de um modelo de contexto (por exemplo, o tamanho máximo do contexto) a partir do teste;
- ajustar pré-processamento (imputação, codificação de categorias, escalonamento) com estatísticas calculadas sobre o dataset inteiro, e não apenas sobre o treino dentro de `fit`;
- trocar datasets da seleção, ou o subconjunto de datasets dos estudos, depois de ver resultados;
- usar os resultados por dataset do *leaderboard* do TabArena (desempenho de modelos em cada *task*), ou os *runs* e avaliações publicados no OpenML para essas *tasks*, para escolher datasets ou para definir faixas do espaço de busca: esses resultados vêm das mesmas partições cujo teste o projeto usa na avaliação final;
- repetir a avaliação final após ver o teste (apagar `results/test.csv`, alterar o código e executar de novo, ou recongelar e reavaliar), ainda que para "corrigir" um resultado ruim;
- usar um estudo feito no treino de um *fold* externo para alterar o sistema avaliado nos outros *folds*: o treino do *fold* externo 0 contém as instâncias de teste dos *folds* 1 e 2.

O vazamento de dados é inaceitável neste projeto: ele anula os critérios de implementação e de interpretação da Etapa 2 listados na rubrica (ver também a Seção 3.6).

3.3 Validação cruzada aninhada na pós-graduação

O projeto usa validação cruzada aninhada (Varma e Simon, 2006), ilustrada na Figura 2:

- **CV externa**: os 3 *folds* oficiais do OpenML (repetição 0) de cada dataset. Cada instância pertence ao teste de exatamente um *fold* externo. O teste de cada *fold* externo estima o desempenho e é usado uma única vez, por `final_test`, depois do congelamento.
- **CV interna**: 3 *folds* estratificados (semente 42) **dentro do treino** de cada *fold* externo. Nela ficam a busca de hiperparâmetros e **todos** os estudos: sensibilidade, estudo específico do modelo, curvas de complexidade e análise de overfitting.

O orçamento da busca do modelo do grupo (`group_tuned`) é igual para todos os grupos:

- Optuna (otimização bayesiana com o amostrador TPE, semente fixa), **20 tentativas**;
- validação cruzada interna estratificada de **3 folds**, maximizando a AUC OvO;

- limite de **600 segundos** de busca por *fold* externo, verificado entre tentativas: a tentativa em andamento quando o limite é atingido é concluída, de modo que o tempo efetivo de busca (`tune_time_s`) pode exceder 600 s e deve ser relatado;
- quando o treino do *fold* externo excede **10.000 instâncias**, a busca e a CV interna usam uma subamostra estratificada de 10.000 instâncias desse treino (o modelo final é treinado com todo o treino do *fold* externo).

O tempo efetivo de busca é medido e somado ao custo do sistema `group_tuned`. A busca é refeita, de forma independente, no treino de cada *fold* externo; por isso a estimativa no teste externo inclui a variabilidade da própria seleção de hiperparâmetros. Os estudos são **diagnósticos**: seus resultados são relatados e interpretados, mas não alteram o espaço de busca, o *wrapper* nem a configuração padrão, que são justificados a priori. Os *notebooks* executam os estudos no treino do *fold* externo 0 (variável `OUTER_FOLD` dos *notebooks* 03 e 05); o grupo pode repeti-los no treino dos demais *folds* externos, nunca no teste.

`validation.csv` é apenas diagnóstico. O treino de um *fold* externo contém o teste dos outros dois. Por isso, os resultados da fase de desenvolvimento (`validation.csv`, `validation_train.csv`, estudos e curvas) **não** são base de decisões: os cinco sistemas, o controle `group_tuned`, o *wrapper*, a configuração padrão, o espaço de busca e o orçamento são fixados a priori. Depois de `run_validation`, só são permitidas correções de **falha de execução** (linhas com `status = error`, violação do contrato do estimador, erro de instalação), registradas no relatório com a causa e a alteração feita; mudanças motivadas por desempenho não são permitidas.

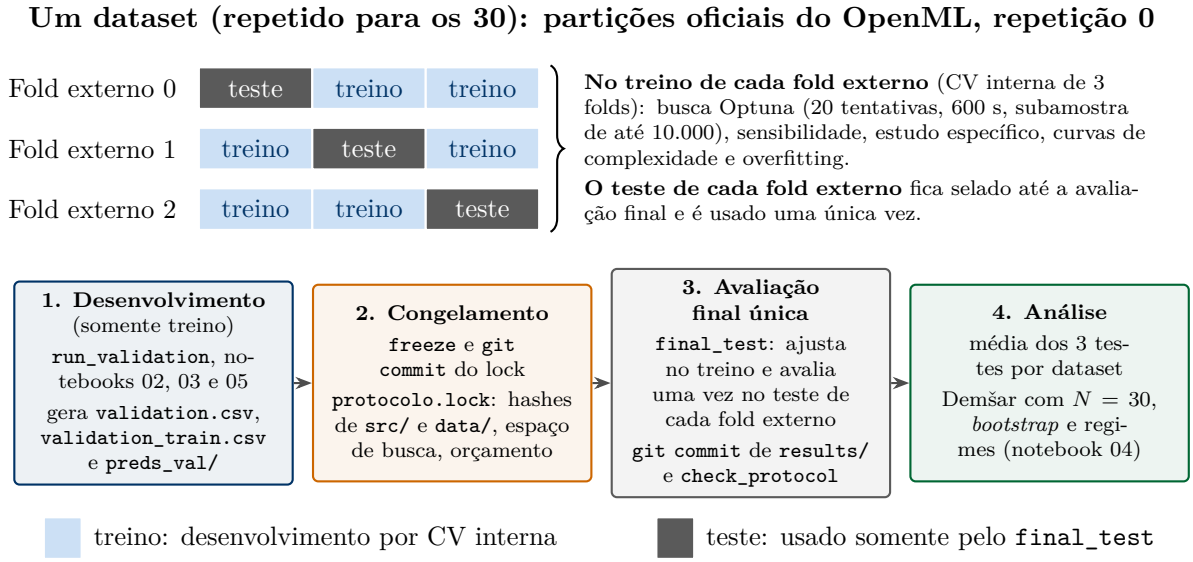


Figura 2: Fluxo do protocolo da pós-graduação para cada dataset. Os 3 *folds* oficiais formam a CV externa; a busca e todos os estudos usam CV interna no treino; o protocolo é congelado; o teste de cada *fold* externo é avaliado uma única vez; a análise de Demšar usa a média dos 3 testes de cada dataset ($N = 30$). A disposição dos blocos é esquemática: as partições oficiais são estratificadas e aleatórias.

3.4 Decisões declaradas a priori

Antes de qualquer resultado, o grupo registra no repositório (e depois no relatório):

- o *wrapper*, a configuração padrão e o **espaço de busca** (`search_space`), com faixas e escalas justificadas pela literatura (artigo original, documentação do pacote, espaços de busca publicados pelo TabArena), **sem** usar os resultados por dataset do *leaderboard* do TabArena; a origem das faixas é discutida no relatório como ameaça à validade;
- a seleção dos 30 datasets (`data/selecao_datasets.txt`);
- o **subconjunto de datasets** dos estudos de sensibilidade e das curvas de complexidade (sugestão: um por regime de tamanho, como no *notebook* 05) e as **grades de valores** de cada estudo, com o sentido de simplicidade de cada hiperparâmetro;
- a métrica primária (AUC OvO) e o nível de significância ($\alpha = 0{,}05$).

Essas decisões são registradas no arquivo `docs/declaracoes_a_priori.md` do repositório, comitado antes de executar o *notebook* 03 e o `run_validation`; o relatório cita o *hash* desse *commit*. O *notebook* 03 é demonstrativo e só deve ser executado depois desse *commit*. A **validação cruzada é obrigatória em todo estudo**: nenhum estudo pode usar uma única partição treino e validação, nem o teste.

3.5 Passo a passo da execução

Todos os comandos são executados na raiz do repositório (prefixo `uv run`, isto é, digite `uv run python -m ...` em cada comando abaixo). As opções completas estão em `docs/INFRAESTRUTURA.md`.

1. **Ambiente e seleção.** `uv sync --extra baselines --extra <modelo>` (para TabM, RealMLP, FT-Transformer, xRFM e TabR, basta `--extra baselines`); preencher `data/selecao_datasets.txt` e conferir com `python -m src.pipeline.check_selection`.
2. **Desenvolvimento.** `python -m src.pipeline.run_validation` avalia os cinco sistemas por CV interna no treino de cada *fold* externo. O `load_task` carrega o dataset inteiro, mas o comando usa apenas os índices de treino (`get_train_indices`): as linhas de teste não participam de nenhum ajuste nem de nenhuma predição antes do `final_test`. Grava `results/validation.csv` (uma linha por dataset, *fold* externo, *fold* interno e sistema, com métricas de validação, tempos, `tune_best_score`, `optimistic`, `params` e `status`), `results/validation_train.csv` (métricas do mesmo estimador no treino interno, medidas numa subamostra estratificada de até 2.000 instâncias) e `results/preds_val/` (probabilidades na validação interna, base da matriz de confusão). Para `group_tuned`, a busca é feita no treino do *fold* externo e os hiperparâmetros escolhidos são reavaliados nas mesmas partições internas: essas linhas têm `optimistic = True`, e o `tune_best_score` é uma estimativa otimista. Se a execução for interrompida, `--resume` continua do ponto em que parou; sem essa opção, o comando recusa sobrescrever um `validation.csv` existente.
3. **Estudos.** Em *notebook*, com os índices de treino de `get_train_indices`: estudo de sensibilidade e estudo específico com `cv_grid` (*notebook* 03) e estudo de overfitting com `validation_curve`, `select_complexity`, `generalization_gap`, `confusion_table` e `recall_per_class` (*notebook* 05). As partições internas (`inner_cv_splits`) e a subamostra (`stratified_subsample`) são as mesmas da busca. Os estudos são diagnósticos e não alteram o protocolo.

4. **Congelamento.** Antes do congelamento, registre em um *commit* todo o código e a seleção (`git add -A` e `git commit`), sem alterações pendentes em `src/` e `data/`; se o `freeze` avisar que há alterações não registradas, faça o *commit* e execute `freeze --force` antes de registrar o lock. `python -m src.pipeline.freeze` grava `results/protocolo.lock` com o sha256 de **todos** os arquivos versionados em `src/**/*.py` e `data/*.py` e de `data/selecao_datasets.txt` (com quebras de linha normalizadas), a assinatura do espaço de busca, o orçamento (tentativas, limite de tempo, subamostra), a semente, os sistemas, os *folds*, os datasets e o *commit* git. Antes do teste, `freeze --force` recongela; havendo qualquer vestígio de avaliação final, o comando recusa recongelar.

5. **Registro do lock.** `git add results/protocolo.lock` e `git commit`, **antes** da avaliação final. O *commit* do lock é a evidência de que o protocolo foi fixado antes de o teste ser usado. Em seguida, envie o *commit* ao GitHub (`git push`), o que registra a ordem dos *commits*.

6. **Avaliação final, uma única vez.** `python -m src.pipeline.final_test` (ou o *notebook* 06) ajusta cada sistema no treino de cada *fold* externo (com a busca refeita nesse treino, para `group_tuned`) e avalia uma vez no teste. Orçamento, semente, sistemas, *folds* e datasets vêm do lock, não da linha de comando. O comando só executa se: (a) o diretório é um repositório git; (b) não há alterações não comitadas em `src/`, `data/` nem no lock; (c) o lock está versionado e idêntico ao de `HEAD`; (d) os hashes do lock conferem com o repositório; (e) não há vestígio de avaliação anterior (`test.csv`, `test_log.csv`, `test_manifest.json`, `preds/` ou *commit* com esses resultados). A opção `--resume` completa somente as combinações que o manifesto registra como pendentes (interrupção) ou com erro, sob o mesmo lock, e recusa se alguma linha concluída de `test.csv` faltar ou tiver sido alterada. Gera `results/test.csv`, `results/preds/`, `results/test_log.csv` (eventos `inicio`, `sistema`, `fim` e `recusa`, com o sha256 do lock e o *commit* `git_head`; retomadas marcadas com `resume=True`) e `results/test_manifest.json` (lock usado, `status` de cada combinação: pendente, ok ou error; `rows`: hash de cada linha ok de `test.csv`; `runs`: execuções com `git_head`). CSV e JSON são gravados com terminador de linha `\n`.

7. **Registro dos resultados.** `git add results` e `git commit` **imediatamente** após o `final_test`, com `git push` logo em seguida.

8. **Conferência.** `python -m src.pipeline.check_protocol` verifica no histórico git que o *commit* do lock antecede o primeiro *commit* com resultados de teste, que houve uma única avaliação inicial (retomadas apenas de pendências ou erros), que os hashes do lock conferem com os arquivos no *commit* do lock, no *commit* da avaliação e em `HEAD`, e que nenhum *commit* posterior altera arquivos travados sem justificativa. A justificativa é registrada em uma linha iniciada por `Justificativa:` na mensagem do *commit* que altera o arquivo travado (por exemplo, com um segundo `-m`); mesmo justificada, a alteração aparece como aviso no `check_protocol`, é examinada na correção e deve ser explicada no relatório. A mesma ferramenta é usada na correção.

9. **Análise.** *Notebook* 04 sobre `results/test.csv`: Demšar, *bootstrap*, regimes e custo. Nada do que aparece nele retroalimenta o modelo.

As travas locais de `freeze` e `final_test` são **dissuasórias**: quem controla o diretório pode apagar arquivos. A evidência do protocolo é o **histórico git**; por isso o lock é comitado antes da avaliação final e `results/` logo depois, e o histórico não deve ser reescrito (*rebase*, *amend* ou *push* forçado) a partir desse ponto.

Ordem dos *notebooks*: 01 (catálogo e seleção), 02 (baselines por CV interna), 03 (modelo do grupo, busca curta e sensibilidade), 05 (overfitting), 06 (avaliação final, após o `freeze`) e 04 (análise estatística, após o `final_test`). Os *notebooks* 02, 03 e 05 usam somente o treino de um *fold* externo. O espaço de busca (`src/models/group_model.py`) e as declarações a priori (`docs/declaracoes_a_priori.md`) são comitados antes de executar o *notebook* 03, que é demonstrativo e não motiva mudanças no espaço de busca; `results/validation.csv` e `results/validation_train.csv` são comitados logo após `run_validation`.

Tabela 1: Arquivos gerados pelo protocolo no diretório `results/` (versionados no repositório do grupo e entregues, exceto `preds_val/`).

Arquivo	**Gerado por**	**Conteúdo**
`validation.csv`	`run_validation`	métricas de validação interna por (dataset, *fold* externo, *fold* interno, sistema); diagnóstico (não é base de decisões) e base da lacuna de generalização
`validation_train.csv`	`run_validation`	métricas no treino interno (subamostra de até 2.000 instâncias, coluna `n_train_eval`)
`preds_val/`	`run_validation`	probabilidades na validação interna (não versionadas; regeneráveis apenas antes da avaliação final, pois `run_validation` recusa executar no diretório de resultados depois dela; conclua o *notebook* 05 antes do `freeze`), base da matriz de confusão
`figures/`	*notebooks* 04 e 05	curvas de validação, lacunas, matrizes de confusão, diagramas CD, *boxplots*, custo
`protocolo.lock`	`freeze`	hashes de `src/` e `data/`, espaço de busca, orçamento, semente, sistemas, *folds*, datasets, *commit*
`test.csv`	`final_test`	uma linha por (dataset, *fold* externo, sistema): métricas de teste, tempos e `status`
`preds/`	`final_test`	probabilidades de teste (`<task>_fold<f>_<sistema>.npz`), base do *bootstrap*
`test_log.csv`	`final_test`	eventos (início, combinação, fim, recusa): horário, sha256 do lock, `git_head`, combinação, `status` e detalhe
`test_manifest.json`	`final_test`	sha256 do lock, `status` e hash (`rows`) de cada combinação, execuções (`runs`, com `git_head`)

3.6 Regras e penalidade

- O teste de cada *fold* externo é usado somente por `final_test`, uma única vez, depois do congelamento e do *commit* do lock. Execuções com `--resume` são aceitas apenas para completar pendências ou erros e devem ser explicadas no relatório.
- `validation.csv` é apenas diagnóstico: depois de `run_validation`, só se admitem correções de falha de execução, registradas no relatório; mudanças motivadas por desempenho não são permitidas.
- É proibido usar os resultados por dataset do *leaderboard* do TabArena (ou os *runs* e avaliações publicados no OpenML para essas *tasks*) na seleção dos datasets e na definição do espaço de busca.
- Qualquer uso do teste antes do *commit* do lock, mais de uma avaliação final, reexecução após apagar resultados ou alteração de arquivos travados depois do teste sem justificativa **anula os critérios de implementação e de interpretação** da Etapa 2 (ver a rubrica). A verificação usa `check_protocol`, o histórico git, `results/test_log.csv`, `results/test_manifest.json` e `results/protocolo.lock`.

Reprovação em `check_protocol` é tratada como uso indevido do teste, salvo explicação técnica aceita.

- Resultados ruins no teste são resultados: o relatório os apresenta tal como obtidos e os discute. Ajustar o sistema depois de ver o teste não é permitido.
- O orçamento de busca do lock deve ser o padrão do enunciado; alterações só com justificativa explícita no relatório.

4 Estudo de overfitting e viés-variância

4.1 Viés, variância e lacuna de generalização

O erro de generalização de um modelo combina **viés** (erro sistemático de um modelo simples demais para o padrão dos dados) e **variância** (sensibilidade do modelo ajustado à amostra de treino, típica de modelos complexos demais para a quantidade de dados). Aumentar a complexidade reduz o viés e aumenta a variância (Hastie, Tibshirani e Friedman, 2009, Seções 2.9 e 7.3). O sintoma observável da variância alta é o **sobreajuste** (*overfitting*): o desempenho no treino continua melhorando enquanto o desempenho na validação estaciona ou piora.

O estudo usa a **lacuna de generalização** orientada:

$$\text{lacuna} = \begin{cases} \text{métrica no treino} - \text{métrica na validação}, & \text{se maior é melhor}, \\ \text{métrica na validação} - \text{métrica no treino}, & \text{se menor é melhor}, \end{cases}$$

em que AUC OvO, ACC e G-Mean são métricas em que maior é melhor e a entropia cruzada (CE) é uma métrica em que menor é melhor. Assim, lacuna > 0 indica overfitting, qualquer que seja a métrica. A lacuna é calculada para **todos os sistemas** (`group_default`, `group_tuned`, LightGBM, XGBoost e CatBoost), pareando cada linha de `results/validation.csv` com a linha correspondente de `results/validation_train.csv` (mesmo dataset, *fold* externo, *fold* interno e sistema). As métricas de treino são medidas numa subamostra estratificada de até 2.000 instâncias do treino interno, o que limita o custo sem alterar o estimador ajustado. O *notebook* 05 apresenta a lacuna geral e por regime de tamanho.

4.2 Curvas de validação de complexidade

Uma **curva de validação** mostra a métrica média de treino e de validação, com a dispersão entre *folds* internos, em função de um hiperparâmetro que controla a complexidade. O grupo traça duas famílias de curvas, por CV interna no treino de um *fold* externo (o *notebook* usa o *fold* externo 0), em cada dataset do subconjunto declarado (sugestão: um por regime de tamanho):

1. um hiperparâmetro de complexidade do **modelo tema** (por exemplo, número de camadas, largura, número de vizinhos, regularização, número de interações), com grade e sentido de simplicidade declarados a priori;
2. `num_leaves` do **LightGBM**, com a grade 2, 4, 8, 16, 32, 64 e 128 já definida no *notebook* 05.

4.3 Regra de 1 erro padrão

Com K *folds* internos, cada valor v da grade tem média $\bar{m}(v)$ e erro padrão $\text{EP}(v) = s(v)/\sqrt{K}$, em que $s(v)$ é o desvio amostral entre *folds*. Seja $v^\star$ o valor de melhor média. A **regra de 1**

erro padrão (Breiman et al., 1984; Hastie, Tibshirani e Friedman, 2009, Seção 7.10) escolhe o valor **mais simples** cuja média está a até 1 EP da melhor:

$$v_{1\text{EP}} = \text{mais simples } v \text{ tal que } \bar{m}(v) \geq \bar{m}(v^\star) - \text{EP}(v^\star) \quad \text{(se maior é melhor)}.$$

A justificativa é que diferenças menores que um erro padrão não são distinguíveis do ruído da própria estimativa; entre modelos indistinguíveis, o mais simples tende a ter menor variância. A Figura 3 ilustra a leitura de uma curva de validação.

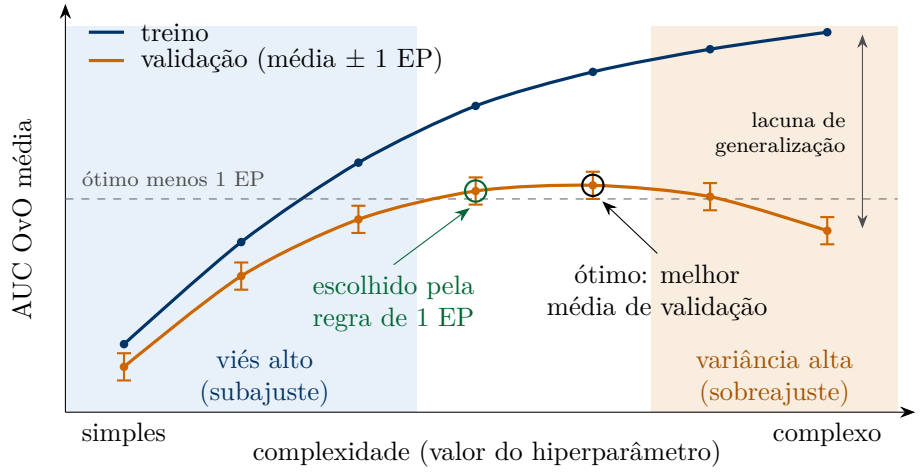


Figura 3: Leitura ilustrativa de uma curva de validação (valores fictícios). O desempenho de treino cresce com a complexidade; o de validação cresce, atinge um máximo e depois cai. À esquerda, o modelo é simples demais (viés alto); à direita, a distância entre treino e validação aumenta (variância alta). O valor de melhor média é o ótimo; a regra de 1 EP escolhe o valor mais simples cuja média está acima da linha tracejada.

4.4 Erros por classe nas predições de validação

A lacuna e as curvas resumem o desempenho em um número; a **matriz de confusão** mostra onde os erros acontecem. O estudo usa as predições de **validação interna** gravadas por `run_validation` em `results/preds_val/` (nunca predições de teste): para cada dataset do subconjunto declarado, o grupo apresenta a matriz de confusão normalizada por linha (a diagonal é o *recall* de cada classe) e o *recall* por classe dos cinco sistemas, discutindo classes raras e a relação com o G-Mean.

4.5 Funções implementadas pelo grupo

O estudo usa funções que o grupo implementa, verificadas por testes de aceitação visíveis (`tests/aceitacao/test_overfitting.py`) e por testes ocultos na correção:

- `generalization_gap(raw, raw_train, metric, higher_is_better)` em `src/pipeline/overfitting.py`: lacuna orientada por sistema (pares, média de treino, média de validação, média e desvio da lacuna);
- `validation_curve(factory, param, values, X, y, folds, seed)`: métricas de treino e de validação para cada valor e cada *fold* interno informado (pelo menos 2 *folds*);
- `select_complexity(curve, metric, rule, simpler_is_lower, higher_is_better)`: escolha pela melhor média (`"best"`) ou pela regra de 1 EP (`"1se"`), com pelo menos 2 *folds* por valor;

- `confusion_table(y_true, y_proba, classes, normalize)` e `recall_per_class(y_true, y_proba, classes)` em `src/pipeline/class_errors.py`: matriz de confusão (contagens ou taxas por linha) e *recall* por classe;
- `plot_validation_curve`, `plot_generalization_gap` e `plot_confusion` em `src/reports/plots_overfitting.py`: curva com faixa de média ± 1 EP entre *folds* (como na Figura 3), barras da lacuna por sistema e mapa de calor da matriz de confusão.

A especificação completa (colunas, ordenação, desempates, exceções) está nas *docstrings*; a API é idêntica à da graduação (IF699 e CIN0209). A infraestrutura pronta fornece `inner_cv_splits` e `stratified_subsample` em `src/pipeline/cv_grid.py`, e `cv_grid`, que prepara a subamostra e as partições internas e delega o laço a `validation_curve`.

4.6 Cautelas na interpretação

- A métrica de treino é **otimista** por construção: ela mede o ajuste aos dados vistos, não a generalização. Só a validação (e, ao final, o teste) estima o desempenho.
- Em **modelos de contexto** (TabPFN-3.5, TabICL v2, TabDPT-1.3) as instâncias de treino fazem parte do contexto, e em **modelos de vizinhança** (ModernNCA, TabR) cada instância de treino é recuperável como vizinha de si mesma. Nesses casos a métrica de treino é quase trivial e a lacuna reflete o mecanismo do modelo, não necessariamente sobreajuste; a interpretação deve se apoiar sobretudo na curva de validação.
- As linhas de `group_tuned` em `validation.csv` são otimistas (`optimistic = True`), pois os hiperparâmetros foram escolhidos nas mesmas partições internas; a lacuna desse sistema tende a ser subestimada.
- As curvas usam um único *fold* externo e poucos datasets: são evidência descritiva. O valor escolhido pela regra de 1 EP **não** é usado para alterar o espaço de busca nem o lock; ele é comparado, no relatório, com a faixa declarada a priori.
- A busca com limite de tempo não é determinística: com a mesma semente, o número de tentativas concluídas e `tune_time_s` podem variar entre máquinas e execuções.
- Quando o treino excede 10.000 instâncias, os hiperparâmetros são escolhidos numa subamostra e aplicados ao treino completo; a escolha pode não ser ótima para o treino completo, sobretudo em hiperparâmetros que dependem do tamanho da amostra (regularização, tamanho do contexto, número de vizinhos).
- Com $K = 3$ *folds* internos, cujos treinos se sobrepõem, o erro padrão $s(v)/\sqrt{K}$ é ruidoso e tende a subestimar a incerteza; a regra de 1 EP fica, por isso, menos conservadora do que parece.

5 Modelos Selecionados

A seguir os modelos tema recentes a serem atribuídos aos grupos. Todos possuem repositórios públicos ativos e instalação reproduzível por meio de um *extra* do `pyproject.toml` do repositório-template:

1. **TabPFN-3.5 (Tabular Prior-Data Fitted Network, versão 3.5)**, *extra* `tabpfn`
   https://github.com/PriorLabs/TabPFN
   Em CPU, usar a variante 3.5-Fast. Os pesos têm licença não comercial: é necessário aceitar a licença em ux.priorlabs.ai e definir a variável de ambiente `TABPFN_TOKEN`.

2. **TabICL v2 (Tabular Foundation Model for In-Context Learning)**, *extra* `tabicl`
   https://github.com/soda-inria/tabicl

3. **TabDPT-1.3 (Tabular Discriminative Pre-trained Transformer)**, *extra* `tabdpt`
   https://github.com/layer6ai-labs/TabDPT

4. **TabM (Parameter-Efficient Ensembling, ICLR 2025)**, via `pytabkit`, *extra* `baselines`
   https://github.com/yandex-research/tabm

5. **RealMLP (NeurIPS 2024)**, via `pytabkit`, *extra* `baselines`
   https://github.com/dholzmueller/pytabkit

6. **FT-Transformer (Feature Tokenizer + Transformer, NeurIPS 2021)**, via `pytabkit`, *extra* `baselines`
   https://github.com/dholzmueller/pytabkit

7. **xRFM (Recursive Feature Machine, 2025)**, via `pytabkit`, *extra* `baselines`
   https://github.com/dmbeaglehole/xRFM

8. **ModernNCA (Modern Neighborhood Component Analysis, ICLR 2025)**, via TALENT, *extra* `modernnca`
   https://github.com/LAMDA-Tabular/TALENT

9. **EBM (Explainable Boosting Machine)**, *extra* `ebm`
   https://github.com/interpretml/interpret

10. **APLR (Automatic Piecewise Linear Regression)**, *extra* `aplr`
    https://github.com/ottenbreit-data-science/aplr

11. **TabR (Retrieval-Augmented Tabular Deep Learning, ICLR 2024)**, via `pytabkit`, *extra* `baselines`
    https://github.com/yandex-research/tabular-dl-tabr

Quatro dos cinco modelos disponíveis no `pytabkit` (TabM, RealMLP, FT-Transformer e xRFM) não aceitam valores ausentes em atributos numéricos e devem ser envolvidos pelo `ImputedClassifier` do repositório, como os baselines. TabDPT-1.3 e ModernNCA não têm interface *scikit-learn* completa, e o TabR do `pytabkit` não aceita valores ausentes nem atributos categóricos; para esses três modelos, o repositório traz esqueletos de *wrappers* que o grupo deve completar. Todo pré-processamento do *wrapper* é ajustado dentro de `fit`, apenas com os dados de treino recebidos.

Equilíbrio de carga entre os modelos. Os grupos de TabDPT-1.3, ModernNCA e TabR implementam o *wrapper* completo do modelo tema. Os grupos cujo modelo já tem API *scikit-learn* nativa (TabPFN-3.5, TabICL v2, TabM, RealMLP, FT-Transformer, xRFM, EBM e APLR) devem apresentar, em compensação:

- um **estudo de sensibilidade a 2 hiperparâmetros** importantes do modelo: grade de valores de cada hiperparâmetro, declarada a priori, avaliada com `cv_grid` por validação cruzada interna no treino de um *fold* externo (o *notebook* 03 usa o *fold* externo 0) de um subconjunto declarado de datasets (sugere-se ao menos um por regime), mantendo os demais na configuração padrão;

- um **estudo específico do modelo**, sobre um aspecto característico da sua arquitetura, com o mesmo protocolo (CV interna no treino), por exemplo: tamanho do contexto ou número de estimadores nos modelos de contexto (TabPFN-3.5 e TabICL v2); limite de interações no EBM e no APLR; número de membros do *ensemble* no TabM.

Os dois estudos são diagnósticos, apresentados no relatório e considerados no critério de implementação do modelo tema da rubrica. O estudo de overfitting (Seção 4) é obrigatório para **todos** os grupos.

Custo computacional. O orçamento de busca é o mesmo para todos os grupos, mas o custo dos modelos difere substancialmente. Sem GPU, são considerados caros: TabPFN-3.5 acima de cerca de 5.000 linhas de treino, ModernNCA, TabR e FT-Transformer nos datasets com cerca de 50.000 linhas ou mais, e redes neurais com busca ampla. Nesses casos, recomenda-se executar no Google Colab com GPU (plano B descrito em `docs/INFRAESTRUTURA.md`), gravando os resultados de forma incremental e retomando execuções interrompidas. Note que o protocolo executa os sistemas duas vezes por *fold* externo (desenvolvimento e avaliação final; no desenvolvimento, um ajuste por *fold* interno, isto é, 3 por *fold* externo; para `group_tuned`, a busca completa é refeita em cada fase); planeje o tempo de acordo. Se o grupo precisar subamostrar o conjunto de contexto de um modelo de contexto em CPU (até 10.000 instâncias), a regra deve ser declarada a priori no relatório, aplicada igualmente a todos os datasets e *folds* e discutida como ameaça à validade. O regime médio, comum a todos os grupos, inclui os dois datasets de maior dimensionalidade do catálogo, Bioresponse (1.776 atributos) e hiva_agnostic (1.617 atributos categóricos, com classe minoritária de 42 instâncias), que tendem a ser os mais caros para modelos de contexto, redes neurais e modelos de vizinhança; meça o tempo neles logo no início.

6 Infraestrutura Recomendada

O ambiente é gerenciado por `uv` com Python 3.12. O núcleo comum (dados, métricas, busca e análise) é instalado por `uv sync`; cada modelo tema tem um *extra* próprio (`baselines`, `tabpfn`, `tabicl`, `tabdpt`, `modernnca`, `ebm`, `aplr`), e o arquivo `uv.lock` fixa as versões exatas. Os principais pacotes são:

- `openml`: carregamento das *tasks* e partições oficiais do TabArena-v0.1;

- `pytabkit`: API *scikit-learn* unificada para LightGBM, XGBoost, CatBoost, TabM, RealMLP, FT-Transformer, xRFM e TabR, com configurações padrão meta-aprendidas;

- `tabpfn`, `tabicl`, `tabdpt`, `interpret`, `aplr` e TALENT: pacotes oficiais dos respectivos modelos tema;

- `optuna`: busca de hiperparâmetros por otimização bayesiana (TPE);

- `scipy`: distribuições e testes usados na implementação do protocolo de Demšar (2006). Bibliotecas que entregam esse protocolo pronto não devem ser usadas na implementação avaliada; podem ser usadas apenas para conferência, com menção no relatório;

- `pytest` e `ruff`: testes automatizados e verificação de estilo, executados também na integração contínua (GitHub Actions), que instala as versões do `uv.lock` com o `torch` para CPU.

7 Entrega e Avaliação

Cada grupo deve entregar:

- Slides utilizados na apresentação sobre o modelo (os dois conjuntos de slides podem compor um único arquivo, como no `slides-template-IN1102.pdf`);
- Slides com resultados detalhados do experimento, incluindo o protocolo (diagrama do fluxo), o estudo de overfitting, tabelas das métricas, resultado do teste de Friedman com Iman-Davenport, diagramas de Bonferroni-Dunn e de Nemenyi (CD), comparação padrão vs. com busca e análise por regime;
- Relatório descrevendo o trabalho realizado e os resultados alcançados, contendo a tabela detalhada e a justificativa dos 30 datasets selecionados, o protocolo efetivamente seguido, o estudo de overfitting, a avaliação final, as respostas às perguntas dos roteiros dos *notebooks* 04 e 05, a análise por regime e as ameaças à validade;
- Código utilizado para realizar os experimentos, no repositório do grupo, com os testes de aceitação passando e os arquivos da Tabela 1 (`validation.csv`, `validation_train.csv`, `protocolo.lock`, `test.csv`, `test_log.csv`, `test_manifest.json`, `preds/` e `figures/`), em ambiente reproduzível (`pyproject.toml` e `uv.lock`; opcionalmente, `Dockerfile`);
- *Model card* detalhado do modelo tema, conforme `model_cards/TEMPLATE.md` do repositório.

O repositório traz materiais de apoio na pasta `entregaveis/`: estrutura sugerida do relatório (`relatorio-template-IN1102.pdf`), estrutura sugerida dos slides (`slides-template-IN1102.pdf`, cerca de 24 *frames* em dois blocos contíguos de 10 minutos) e rubrica de avaliação detalhada (`rubrica-IN1102.pdf`), que decompõe os pesos abaixo em critérios específicos.

A avaliação será baseada em:

- Clareza e profundidade na explicação do modelo (40%);
- Qualidade e rigor nos experimentos realizados (50%), incluindo implementação (verificada por testes de aceitação visíveis e por testes ocultos), protocolo experimental e estudo de overfitting, interpretação estatística e reprodutibilidade;
- Qualidade da apresentação (10%).

Na correção, o código é executado com testes adicionais não publicados e com a variável `AM_STRICT=1`, que desativa a conversão de componentes não implementados em *xfail*. É **proibido alterar as assinaturas** (nomes, parâmetros e tipos de retorno) das funções e classes indicadas como trabalho do grupo, pois os testes dependem delas. Também é proibido alterar os testes visíveis (pasta `tests/`, inclusive `tests/conftest.py`). A análise estatística será conferida com base no `results/test.csv` entregue, e a integridade do protocolo, com `python -m src.pipeline.check_protocol` sobre o histórico git do repositório, além de `protocolo.lock`, `test_log.csv` e `test_manifest.json`.

7.1 Marcos de orientação

Há dois marcos de orientação, em datas divulgadas pelo professor, nos quais os grupos podem buscar orientação presencial ou remotamente. Os marcos não são avaliativos: não alteram os critérios nem os pesos da avaliação.

1. **Marco 1:** após a seleção das bases, a instalação do ambiente e o modelo do grupo passando nos testes de contrato, antes de executar `run_validation`.

2. **Marco 2:** após a validação (`run_validation`) e o estudo de overfitting, antes do `freeze`.

8 Repositório-Template

Os grupos receberão acesso ao repositório-template oficial `projeto-final-IN1102-2026.2`, privado e distribuído sob licença MIT (arquivo `LICENSE`, titular Leandro Maciel Almeida). Cada grupo deve criar o próprio repositório a partir dele com o botão *Use this template* do GitHub, **mantê-lo privado** e conceder acesso ao professor. O template separa:

- **Infraestrutura pronta**, comum a todos os grupos para garantir a comparabilidade: catálogo dos 38 datasets e validador da seleção, carregamento via OpenML com *cache*, partições oficiais (`get_folds` e `get_train_indices`), métricas, baselines com imputação, sistemas comparados (`systems.py`), motor de busca com Optuna, CV interna e estudos de sensibilidade (`cv_grid.py`), fase de desenvolvimento (`run_validation.py`), congelamento (`freeze.py`), avaliação final com guardas, registro e retomada (`final_test.py`), integridade e verificação pelo histórico git (`integrity.py` e `check_protocol.py`), leitura dos resultados com a regra de completude e definição dos regimes;

- **Trabalho do grupo**, marcado no código com `NotImplementedError("TODO(grupo): ...")` e verificado por testes de aceitação: modelo tema e espaço de busca (`src/models/group_model.py` e, para TabDPT-1.3, ModernNCA e TabR, `src/models/wrappers.py`); módulo `src/pipeline/demsar.py`; `bootstrap_ci` em `src/pipeline/bootstrap.py`; agregações por regime em `src/pipeline/regime.py`; análise de overfitting em `src/pipeline/overfitting.py`; erros por classe em `src/pipeline/class_errors.py`; tabelas-resumo e gráficos em `src/reports/` (inclusive `plots_overfitting.py`); e a interpretação, guiada pelas perguntas dos *notebooks* 04 e 05.

Os testes de aceitação aparecem como *xfail* ("ainda não implementado") enquanto o componente correspondente levantar `NotImplementedError`; qualquer outro erro é reportado como falha. No template recém-criado, `uv run pytest` não apresenta falhas: todos os testes de infraestrutura passam, e os testes de aceitação aparecem como *xfail* até que o componente correspondente seja implementado; as contagens exatas da versão distribuída constam da seção *Status do template* do `README.md`. Os testes de infraestrutura incluem testes "espiões" que contam as chamadas de ajuste e de predição e verificam que a busca, `run_validation` (inclusive o comando completo, com os carregadores substituídos) e `cv_grid` nunca usam índices de teste e que `final_test` ajusta somente no treino e faz exatamente uma predição sobre o teste por sistema e *fold*; testes com repositórios git temporários verificam as guardas de `freeze`, `final_test` e `check_protocol`; e uma varredura dos *notebooks* 01 a 05 falha se algum deles referenciar o teste (o *notebook* 04 pode ler `results/test.csv` e as predições de teste após o `final_test`, mas nenhum *notebook* pode usar índices de teste de `get_folds`

nem funções internas do `final_test`). O critério objetivo do módulo de estatística é reproduzir o exemplo da Tabela 6 e os valores críticos da Tabela 5 de Demšar (2006). O arquivo `README.md` do template apresenta uma seção *Status do template* com essas categorias; o documento `docs/INFRAESTRUTURA.md` traz um guia detalhado do ambiente, dos comandos, do uso do Colab com GPU, do *token* do TabPFN e da solução de problemas.

A Anexo: Glossário Técnico

Este anexo apresenta uma referência conceitual breve sobre as ferramentas e os termos técnicos utilizados no projeto. Para guias passo a passo, com comandos e solução de problemas, consulte o arquivo `docs/INFRAESTRUTURA.md` no repositório-template.

Ferramentas

Ambiente virtual (*virtual environment* ou *venv*). Diretório isolado que contém uma instalação Python independente, com suas próprias bibliotecas. Permite que cada projeto tenha versões específicas de pacotes sem conflitar com outros projetos ou com o Python global do sistema.

`uv`. Gerenciador de pacotes e ambientes desenvolvido pela Astral, escrito em Rust. Substitui simultaneamente `pip`, `venv` e `pip-tools`, com instalação muito mais rápida. Lê o `pyproject.toml`, cria o ambiente com `uv sync` e executa comandos dentro dele com `uv run`. É o gerenciador adotado neste projeto.

`pyproject.toml`, *extras* e `uv.lock`. O `pyproject.toml` (PEP 621) descreve metadados e dependências do projeto. Os *extras* são grupos opcionais de dependências instalados sob demanda (por exemplo, `uv sync --extra baselines --extra ebm`). O `uv.lock` registra as versões exatas resolvidas, garantindo que todos os integrantes e a correção usem o mesmo ambiente.

`Dockerfile` e contêineres. Receita textual que descreve como construir uma imagem Docker com o sistema operacional, as dependências e o código do projeto. Permite reproduzir o ambiente em qualquer máquina. Opcional neste projeto.

Integração contínua (GitHub Actions). Execução automática de verificação de estilo e testes a cada *push* no repositório do grupo, definida em `.github/workflows/testes.yml`. O *workflow* exporta as versões do `uv.lock` e instala o `torch` para CPU; o grupo acrescenta o *extra* do seu modelo na linha do `uv export` e, no caso do TabPFN-3.5, cadastra o segredo `TABPFN_TOKEN` no repositório.

Git, repositório e *commit*. O git é um sistema de controle de versões. O **repositório** é a pasta do projeto com todo o seu histórico, e um *commit* é um registro permanente do estado dos arquivos, com autor, data e mensagem: `git add` escolhe os arquivos e `git commit -m` registra o *commit* com uma mensagem. Neste projeto, "comitar" significa registrar em um *commit*. O histórico de *commits* é a evidência do protocolo; por isso, depois do *commit* do lock, não se usa *amend*, *rebase* nem *push* forçado. Instalação, identificação (`git config`) e comandos básicos estão em `docs/INFRAESTRUTURA.md`, seção 1.1.

Terminal e raiz do repositório. O terminal (PowerShell ou Git Bash no Windows; Terminal no macOS e no Linux) é a janela em que os comandos são digitados. A **raiz do repositório** é a pasta principal do projeto, a que contém `pyproject.toml` e `README.md`; todos os comandos do enunciado são executados a partir dela (use `cd` para entrar nela).

GitHub, *clone* e *push*. O GitHub hospeda a cópia remota do repositório do grupo. `git clone` cria a cópia local; `git pull` traz os *commits* dos colegas; `git push` envia os *commits* locais ao GitHub, onde o professor os consulta e a integração contínua os testa. O acesso a repositório privado exige autenticação (Git Credential Manager ou *token* pessoal), descrita em `docs/INFRAESTRUTURA.md`, seção 1.1.

Estimador compatível com *scikit-learn*. Objeto com o método `fit(X, y)`, que ajusta o modelo e devolve o próprio objeto, o método `predict_proba(X)`, que devolve uma probabilidade por classe na ordem de `classes_`, e os métodos `get_params` e `set_params`, de modo que `sklearn.base.clone` consiga copiá-lo. É o contrato verificado por `tests/aceitacao/test_modelo_grupo.py`.

***Wrapper*.** Classe que envolve um modelo cuja interface não segue o padrão do *scikit-learn* e o expõe como estimador compatível, convertendo as entradas (por exemplo, separando atributos numéricos e categóricos) e as saídas. Todo pré-processamento do *wrapper* é ajustado dentro de `fit`, só com os dados de treino.

***Notebook* Jupyter.** Documento interativo que intercala texto e células de código executáveis. Os *notebooks* do projeto ficam em `notebooks/` e são abertos com `uv run jupyter lab`, executado na raiz do repositório.

Hash sha256. Sequência de 64 caracteres calculada a partir do conteúdo de um arquivo; qualquer alteração no arquivo muda o hash. O lock guarda o sha256 dos arquivos travados, e `final_test` e `check_protocol` recalculam esses valores para detectar alterações.

***Seed* (semente aleatória).** Número inteiro fixo passado a geradores pseudoaleatórios para que sorteios (partições, inicialização de pesos, amostragem de hiperparâmetros, *bootstrap*) sejam reproduzíveis.

`pytest`, testes de aceitação e *xfail*. `pytest` é o *framework* de testes automatizados em Python. Testes de aceitação verificam se um componente implementado pelo grupo cumpre a especificação. Um teste marcado como *xfail* é uma falha esperada: no template, indica componente ainda não implementado. Com `AM_STRICT=1`, essa conversão é desativada.

Optuna. Biblioteca para busca de hiperparâmetros baseada em otimização bayesiana (TPE). Mais eficiente que a busca em grade para espaços grandes.

OpenML e TabArena. O OpenML hospeda *datasets* tabulares com identificadores únicos de *task* e partições padronizadas. O TabArena-v0.1 é um benchmark vivo de dados tabulares, com 38 *tasks* de classificação reunidas na suíte 457 do OpenML.

***Model card*.** Documento curto e padronizado que descreve um modelo de aprendizado de máquina: arquitetura, dados, métricas, limitações e considerações éticas (Mitchell et al., 2019).

Protocolo experimental e overfitting

Conjunto de teste (*holdout*). Parte dos dados separada antes de qualquer decisão e usada uma única vez, ao final, para estimar o desempenho de generalização do sistema já escolhido. Neste projeto, o teste de cada *fold* externo oficial; só o comando `final_test` o utiliza.

Conjunto de validação. Dados, separados do teste, usados para escolher hiperparâmetros, variantes e regras do modelo. Como participa das escolhas, a estimativa de desempenho na validação é otimista para o sistema escolhido.

Validação cruzada (*cross-validation*). Técnica para estimar o desempenho sem usar o conjunto de teste final. Os dados são divididos em k partes (*folds*); o modelo é treinado em $k-1$ delas e avaliado na restante, repetindo k vezes. É obrigatória em todos os estudos do projeto.

Validação cruzada externa e interna (aninhada). Na CV aninhada (Varma e Simon, 2006), a CV externa estima o desempenho e a CV interna, executada apenas no treino de cada *fold* externo, faz as escolhas (busca e estudos). Neste projeto, a CV externa são os 3 *folds* oficiais e a CV interna tem 3 *folds* estratificados no treino.

Vazamento de dados (*data leakage*). Qualquer caminho pelo qual informação do teste influencia o sistema avaliado: escolher hiperparâmetros, faixas, grades ou regras olhando o teste, ajustar pré-processamento com o dataset inteiro, repetir a avaliação final após ver o resultado. Produz estimativas otimistas e é inaceitável no projeto.

Congelamento (*lock*). Registro, antes da avaliação final, de tudo o que define o sistema avaliado. O comando `freeze` grava `results/protocolo.lock` com os hashes de todo o código versionado em `src/` e `data/`, da seleção e do espaço de busca e com o orçamento; `final_test` recusa executar se algo divergir ou se o lock não estiver comitado.

Histórico git como evidência. As travas locais podem ser contornadas apagando arquivos; o histórico de *commits* não. O lock comitado antes dos resultados, um único evento de avaliação inicial em `test_log.csv` e a ausência de alterações posteriores nos arquivos travados são conferidos por `check_protocol`.

Matriz de confusão. Tabela que cruza a classe verdadeira (linhas) com a classe prevista (colunas). Normalizada por linha, a diagonal é o *recall* de cada classe; a média geométrica desses *recalls* é o G-Mean.

Lacuna de generalização. Diferença orientada entre o desempenho no treino e na validação (treino menos validação quando maior é melhor; validação menos treino quando menor é melhor). Valores positivos indicam sobreajuste.

Curva de validação. Gráfico do desempenho médio de treino e de validação, com a dispersão entre *folds*, em função de um hiperparâmetro de complexidade. Mostra as regiões de subajuste e de sobreajuste.

Regra de 1 erro padrão (1 EP). Escolhe o valor mais simples cuja média de validação está a até um erro padrão da melhor média (Breiman et al., 1984), preferindo modelos mais simples entre opções estatisticamente indistinguíveis.

Viés e variância. Componentes do erro de generalização: o viés é o erro sistemático de um modelo simples demais (subajuste); a variância é a sensibilidade do modelo à amostra de treino, típica de modelos complexos demais (sobreajuste). Aumentar a complexidade reduz o viés e aumenta a variância.

Estratificação. Divisão dos dados que preserva, em cada parte, a proporção das classes do conjunto original. As partições oficiais, a CV interna e as subamostras do projeto são estratificadas.

Imputação. Preenchimento de valores ausentes. Nos baselines e nos modelos envolvidos pelo `ImputedClassifier`, os ausentes numéricos recebem a mediana calculada no treino, com um indicador binário de ausência; a estatística nunca é calculada com dados de validação ou de teste.

Comparação estatística

Hipótese nula, p-valor e nível de significância. A hipótese nula (H_0) afirma que não há diferença entre os sistemas comparados. O p-valor é a probabilidade, sob H_0, de observar uma estatística tão extrema quanto a obtida ou mais. Rejeita-se H_0 quando o p-valor é menor que o nível de significância α, fixado a priori em 0,05. Não rejeitar H_0 não prova que os sistemas são equivalentes.

Postos (*ranks*) e posto médio. Em cada dataset, os k sistemas são ordenados pela métrica (1 = melhor); empates recebem a média dos postos que ocupariam. O posto médio R_j do sistema j é a média dos seus postos nos N datasets.

Teste de Friedman. Teste não paramétrico para comparar k sistemas em N datasets. Sob a hipótese nula de equivalência, os postos médios devem ser iguais; a estatística é

$$\chi_F^2 = \frac{12N}{k(k+1)} \left[\sum_j R_j^2 - \frac{k(k+1)^2}{4} \right],$$

com $k-1$ graus de liberdade. A aproximação pela distribuição qui-quadrado é indicada para $N > 10$ e $k > 5$ (Demšar, 2006); com $k = 4$, como no teste principal, ela é apenas aproximada, o que reforça o uso de F_F.

Correção de Iman-Davenport. Como χ_F^2 é conservadora, usa-se

$$F_F = \frac{(N-1)\chi_F^2}{N(k-1) - \chi_F^2},$$

com distribuição F de $k-1$ e $(k-1)(N-1)$ graus de liberdade. É o critério de decisão do projeto.

Diferença crítica (CD). Diferença mínima entre postos médios para que dois sistemas sejam considerados diferentes: $\mathrm{CD} = q_\alpha \sqrt{k(k+1)/(6N)}$, em que q_α depende do procedimento (Tabela 5 de Demšar, 2006). O diagrama CD mostra os postos médios em um eixo e liga os sistemas sem diferença significativa. Os pós-testes só são interpretados como testes quando o teste de Friedman (por F_F) rejeita a hipótese nula.

Teste de Nemenyi. Pós-teste para comparar todos os pares de sistemas, com q_α baseado na amplitude studentizada. É conservador, pois controla o erro para $k(k-1)/2$ comparações.

Teste de Bonferroni-Dunn. Pós-teste para comparar um sistema de controle com os demais ($k-1$ comparações), com q_α obtido da distribuição normal com correção de Bonferroni. No diagrama, marca-se o intervalo de $\pm$CD em torno do posto do controle.

Procedimento de Holm. Ajuste de múltiplas comparações em etapas (*step-down*): os m p-valores são ordenados do menor para o maior, $p_1 \leq \cdots \leq p_m$, e p_i, para $i = 1, \ldots, m$, é comparado com $\alpha/(m-i+1)$, interrompendo-se na primeira hipótese não rejeitada. É mais poderoso que Bonferroni-Dunn. No projeto, os p-valores vêm das estatísticas $z = (R_i - R_c)/\sqrt{k(k+1)/(6N)}$ do controle c contra cada sistema i.

Teste de Wilcoxon (*signed-ranks*). Teste não paramétrico para dois sistemas em N datasets: ordena os valores absolutos das diferenças, soma os postos das diferenças positivas (R^+) e negativas (R^-) e usa $T = \min(R^+, R^-)$. No projeto, compara a variante padrão com a variante com busca de hiperparâmetros do modelo do grupo; relate o número de diferenças efetivamente usado.

Intervalo de confiança por *bootstrap*. Reamostragem com reposição das instâncias de teste para estimar a variabilidade da métrica; o intervalo percentil de 95% é dado pelos quantis 2,5% e 97,5% das reamostras. Cada intervalo refere-se a um dataset e *fold*; a média dos limites entre datasets não é um intervalo de confiança da média entre datasets.

ROPE (*Region Of Practical Equivalence*), opcional. Intervalo em torno de zero dentro do qual a diferença entre dois modelos é considerada irrelevante na prática. Usado na análise bayesiana (Benavoli et al., 2017), extensão opcional deste projeto, permite afirmar que dois modelos são praticamente equivalentes, algo que o teste de hipótese clássico não permite.

Referências

- Breiman, L., Friedman, J. H., Olshen, R. A. e Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
- Cawley, G. C. e Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, p. 2079 a 2107.
- Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, p. 1 a 30.
- Benavoli, A., Corani, G. e Mangili, F. (2016). Should We Really Use Post-Hoc Tests Based on Mean-Ranks? *Journal of Machine Learning Research*, 17(5), p. 1 a 10.
- Benavoli, A., Corani, G., Demšar, J. e Zaffalon, M. (2017). Time for a Change: a Tutorial for Comparing Multiple Classifiers Through Bayesian Analysis. *Journal of Machine Learning Research*, 18(77), p. 1 a 36.
- Hastie, T., Tibshirani, R. e Friedman, J. (2009). *The Elements of Statistical Learning*, 2.ª ed. Springer. Seções 2.9, 7.2, 7.3 e 7.10.

- Holzmüller, D., Grinsztajn, L. e Steinwart, I. (2024). Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data. *NeurIPS*.
- Erickson, N. et al. (2025). TabArena: A Living Benchmark for Machine Learning on Tabular Data. *NeurIPS*. https://tabarena.ai
- Mitchell, M. et al. (2019). Model Cards for Model Reporting. *FAT**.
- Varma, S. e Simon, R. (2006). Bias in Error Estimation When Using Cross-Validation for Model Selection. *BMC Bioinformatics*, 7, 91.